TECHNICAL FIELD GUIDE

Agilex 5 E-Series 065B 4Kp30 Camera + AI

Field-proven deployment, AI model compilation, and validation procedure for the Altera Agilex 5 FPGA E-Series 065B Modular Development Kit.

Revision 4.0 | 4 September 2026

First-Time Bring-Up and Customer Demonstration Runbook

Document control

Item	Value
Document type	First-time provisioning and offline customer-demo runbook
Audience	New hardware, FPGA, embedded-software, validation, and field engineers
Status	Field-validated pre-built release procedure
Validated configuration	Agilex 5 E-Series 065B MDK; one or two Framos FSM:GO IMX678C cameras
Release assets	`altera-fpga/agilex-ed-camera-ai`, tag `rel-25.1`
Documentation baseline	Official Altera camera guide `rel-25.1`; user-supplied current guide `rel-26.1`

Purpose and scope

Use this guide to install and validate the 4Kp30 Multi-Sensor Camera with AI Inference Solution System Example Design on an Altera Agilex™ 5 FPGA E-Series 065B Modular Development Kit. It is written for a first-time operator and contains two paths:

- **First-time provisioning:** downloads assets, writes the microSD card, programs QSPI, and compiles/installs the AI models. Internet access is required.
- **Customer demonstration:** boots a prepared board and prepared microSD card using only files carried on the host laptop. Internet access is not required at the customer site.

Compatibility notice: Although the documentation is published under `rel-26.1`, this pre-built design's binaries and source release are `rel-25.1`. Use matching assets. Quartus Pro 25.1 is required to rebuild the design. A compatible newer Quartus Programmer can program the pre-built `.jic`; verify that it detects the carrier board before programming.

Before you begin

- Confirm that the host is Linux x86_64 with Quartus Programmer, a serial-terminal application, and a web browser.
- For first-time provisioning, also confirm Internet access, Docker, and at least 8 GB of free RAM.
- Ensure that the cameras, microSD card, J35/J2 USB cables, Ethernet, DP cable, and display are available.
- Keep the board powered off when connecting camera cables or inserting/removing the microSD card.
- Obtain approval for the Ultralytics model license before downloading or compiling the YOLOv8 models.

Customer demonstration quick-start — no Internet required

Use this page at the customer site. It starts a design that was prepared in advance. Internet access is **not** needed at the customer site; local Ethernet is still required for the browser UI.

> **Offline kit requirement:** Carry the prepared microSD card, local copy of `top.core.jic`, this PDF, a Linux laptop with Quartus Programmer and Minicom already installed, both micro-USB cables, DP cable, Ethernet cable, and the compatible camera modules. The prepared card must contain the Linux image and the four compiled AI-model assets for Detect/Pose.

Decide which customer-site path applies

- If QSPI is known to contain `top.core.jic`, skip directly to **Boot and demonstrate**.
- If the board is new, erased, or its QSPI state is unknown, complete **Program QSPI from the local offline kit** first.

Program QSPI from the local offline kit

1. Power the board **off** and set SOM switch `S4 = OFF-OFF` (JTAG mode).
2. Connect carrier `J35` micro-USB to the laptop and power on the board.
3. From the directory holding the local `top.core.jic`, verify the cable and program QSPI:

HOST TERMINAL COMMANDS

```
quartus_pgm -l
quartus_pgm -c 1 -m jtag -o "pvi;top.core.jic"
```

4. Wait for `Successfully performed operation(s)` and `0 errors`.
5. Power the board **off**, set `S4 = ON-ON` (ASx4/QSPI boot), and keep the prepared microSD card installed.

Boot and demonstrate

1. With the board off, insert the prepared microSD card. Connect both cameras, DP display, `J2` HPS serial USB, and `J6` Ethernet.
2. Set the DP display to its DP input; then power on the board. Leave the display connected throughout boot.
3. Open the HPS serial terminal at 115200 baud, 8 data bits, no parity, one stop bit, with no flow control. On the validated laptop, the HPS UART is `/dev/ttyUSB3`:

HOST TERMINAL COMMANDS

```
gnome-terminal --title="Agilex-HPS-ttyUSB3" -- bash -c \
  "sudo minicom -D /dev/ttyUSB3 -b 115200; exec bash"
```

4. Log in as `root` (no password), then obtain the board's local address:

HOST TERMINAL COMMANDS

```
ifconfig eth0
```

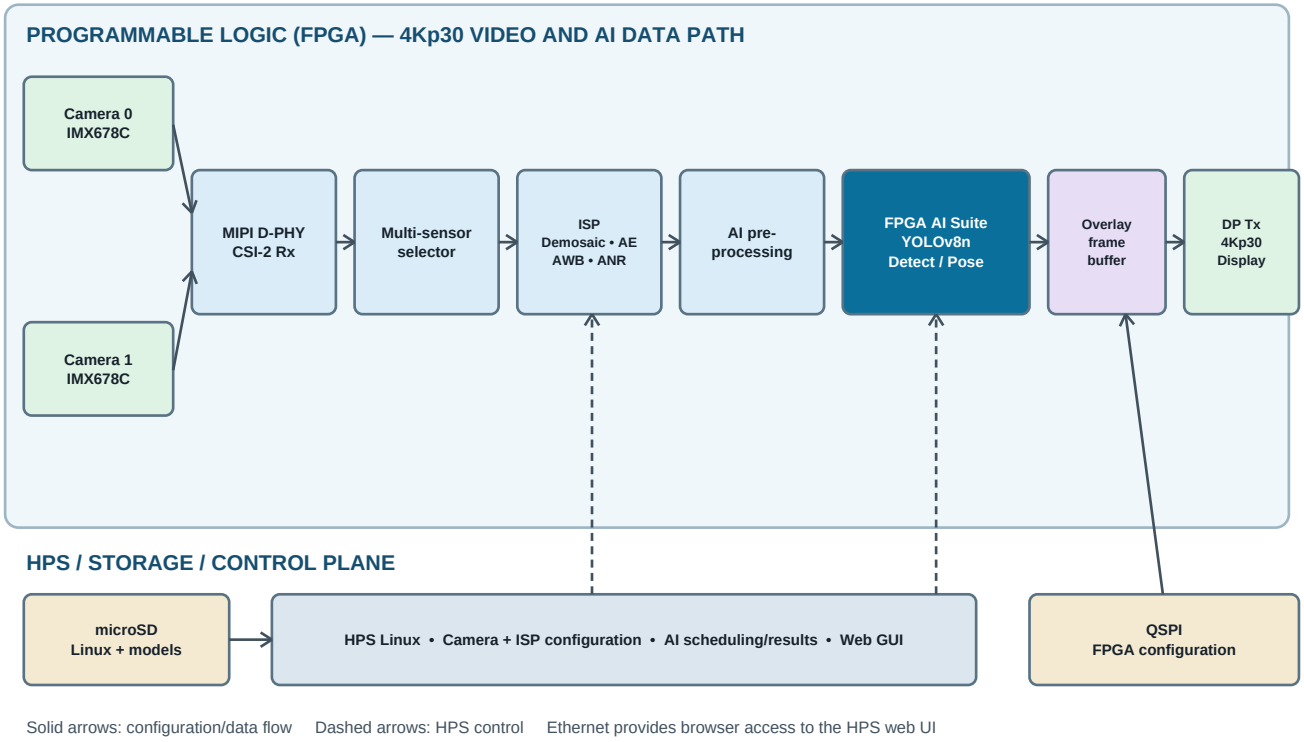
5. In the laptop browser, open `http://BOARD_IP/`, replacing `BOARD_IP` with the `inet addr` value.
6. Confirm DP video, select camera 0 and camera 1 in the UI, and verify live video from each.
7. Select **Detect** to demonstrate object boxes/labels, then **Pose** to demonstrate pose overlays.

If the demonstration does not start

- **No serial output:** Do not assume `/dev/ttyUSB3` on every laptop. Disconnect/reconnect only `J2`; its four reappearing UARTs form the J2 group. Use its third port, as described in section 6.
- **No web UI, DP signal, or AI overlay:** Internet is unnecessary, but the board needs a DHCP address, display input/DP cable, and prepared model files. Use the detailed troubleshooting table.

Implementation block diagram

The video path runs in programmable logic (PL). Linux on the HPS configures the camera/ISP/AI components, schedules inference, processes AI results, and exposes the web UI. The multi-sensor selector chooses the active camera source for the 4Kp30 pipeline.



Data flow: Camera pixels travel left-to-right through MIPI reception, source selection, ISP, AI pre-processing, FPGA AI Suite, overlay, and DisplayPort. The dashed arrows represent HPS control; QSPI supplies FPGA configuration and microSD supplies Linux plus compiled models.

1. What this guide installs

This procedure writes:

- an HPS-first Yocto Linux image to the microSD card;
- a QSPI configuration image to the board flash;
- YOLOv8 nano detection and pose models to the board's SD-backed root filesystem.

The application starts automatically at boot. The browser UI controls the camera source and AI mode; DP carries the 4K video output with overlays.

2. Required hardware

- Agilex 5 FPGA E-Series 065B Modular Development Kit
- One or two Framos FSM:GO IMX678C cameras with suitable PixelMate MIPI CSI-2 cables
- Minimum 8 GB U3 microSD card and USB card reader
- DP display/cable capable of the required video mode (a 4K-capable display is recommended)
- Micro-USB from carrier **J35** to the host (JTAG/QSPI programming)
- Micro-USB from SOM **J2** to the host (HPS serial console)
- Ethernet from SOM **J6** to a DHCP-enabled network
- Linux x86_64 host with Internet access, Docker, and Quartus Programmer

Safety: Power off the board before inserting the SD card or connecting/reseating the MIPI camera cables. Align pin 1 on each camera cable with pin 1 on its connector.

3. Download and validate the pre-built assets

On the Linux host:

HOST TERMINAL COMMANDS

```
mkdir -p ~/agilex5-camera-4kp30
cd ~/agilex5-camera-4kp30

RELEASE_URL=https://github.com/altera-fpga/agilex-ed-camera-ai/releases/download/rel-25.1
IMAGE=hps-first-vvp-isp-demo-image-agilex5_mk_a5e065bb32aes1.wic.gz
JIC=top.core.jic

curl -fLO "$RELEASE_URL/$IMAGE"
curl -fLO "$RELEASE_URL/$JIC"

IMAGE_SHA256=495605036a85bab7454ae56fabd659a4423a07e256a0ec0cbf4387270f56895c
JIC_SHA256=8dc7434444c276c5b04005d3e664011ec60cc3fa3f07f43eaa3038d0568e7c19
printf '%s  %s\n' "$IMAGE_SHA256" "$IMAGE" | sha256sum -c -
printf '%s  %s\n' "$JIC_SHA256" "$JIC" | sha256sum -c -

gzip -dk "$IMAGE"
```

Both checksum commands must print `OK`. The uncompressed `.wic` is the image written to the entire microSD device.

4. Write the microSD card

Identify the removable microSD device:

HOST TERMINAL COMMANDS

```
lsblk -o NAME,SIZE,MODEL,TRAN,RM,MOUNTPOINTS
```

Use the whole removable disk—for example `/dev/sda`—not a partition such as `/dev/sda1`. Verify the target by model, size, transport, and `RM=1`. Never select the host's NVMe/SATA disk.

Unmount the card's mounted partitions, then write the image:

HOST TERMINAL COMMANDS

```
sudo umount /dev/sdX1
sudo dd if=hps-first-vvp-isp-demo-image-agilex5_mk_a5e065bb32aes1.wic \
  of=/dev/sdX bs=4M conv=fsync status=progress
sync
sudo eject /dev/sdX
```

Replace `sdX` with the confirmed card device. This erases the card. After `eject` completes, insert the card into the SOM microSD slot.

5. Program QSPI

1. Power **off** the board.
2. Set SOM switch **S4 = OFF-OFF** for JTAG mode.
3. Connect carrier **J35** micro-USB to the host and power the board.
4. Confirm that Quartus sees the programming cable:

HOST TERMINAL COMMANDS

```
quartus_pgm -l
```

Expected: an entry similar to `Agilex_5E MDK Carrier`.

5. Program and verify the QSPI device:

HOST TERMINAL COMMANDS

```
cd ~/agilex5-camera-4kp30
quartus_pgm -c 1 -m jtag -o "pvi;top.core.jic"
```

Wait for `Successfully performed operation(s)` and `0 errors`.

6. Power **off** the board, set SOM **S4 = ON-ON** (ASx4/QSPI boot), and leave the microSD installed.

6. Connect, boot, and open the HPS console

With the board powered off, connect:

- both cameras;
- DP display;
- SOM J2 micro-USB;
- SOM J6 Ethernet.

Power on the board. The J2 USB cable exposes four USB serial ports. Its **third** port is the HPS UART. If J35 is also connected, the host can show eight ports, so identify the J2 group by disconnecting and reconnecting only J2:

HOST TERMINAL COMMANDS

```
ls -1 /dev/ttyUSB*
```

The four ports that disappear/reappear belong to J2. Use the third port in that group—e.g., `/dev/ttyUSB6` for group `ttyUSB4` through `ttyUSB7`.

If the host user has no serial-port permission:

HOST TERMINAL COMMANDS

```
sudo usermod -aG dialout "$USER"
```

Sign out and back in for the group change to apply. For an immediate temporary session, use `sudo`.

Open a separate terminal window with Picocom:

HOST TERMINAL COMMANDS

```
gnome-terminal --title="Agilex 5 HPS Console" -- \
  bash -lc 'sudo picocom -b 115200 --flow n /dev/ttyUSB6; exec bash'
```

Use 115200 baud, 8 data bits, no parity, one stop bit, and no flow control. Log in as `root` (no password).

7. Find the board address and open the UI

This image supplies BusyBox `ip`, which does **not** support `ip -br`. At the HPS serial console:

HOST TERMINAL COMMANDS

```
ifconfig eth0
```

Use the `inet addr` value in a host browser:

EXPECTED OUTPUT / REFERENCE

```
http://BOARD_IP/
```

For example, `http://10.42.0.12/`. The IP is DHCP-provided and may change after a reboot. The web UI should load automatically after Linux has booted.

Phase 1 checkpoint

Before compiling AI models, record that the base platform is working:

- QSPI programming completed with `Successfully performed operation(s)`.
- Linux booted from the flashed microSD and accepted the `root` serial login.
- `eth0` received an address and the browser UI opened.
- The DP display detected a signal and showed the selected camera source.

8. Compile YOLOv8 models for the FPGA AI Suite

The pre-built image does not supply YOLO model binaries. Before downloading the models, review and accept the Ultralytics license: https://www.ultralytics.com/license

The release pins PyTorch 2.6.0, which does not provide a Python 3.14 wheel. A host with Python 3.14 will fail with `No matching distribution found for torch==2.6.0+cpu`. The supported, reproducible workaround below uses Ubuntu 22.04 / Python 3.10 inside Docker.

HOST TERMINAL COMMANDS

```
cd ~/agilex5-camera-4kp30
git clone --recurse-submodules -b rel-25.1 \
  https://github.com/altera-fpga/agilex-ed-camera-ai.git source

cd source/yolo_cnn
curl -fLO https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n.pt
curl -fLO https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n-pose.pt

sudo docker run --rm -it \
  -v "$PWD:/work" \
  -w /work \
  ubuntu:22.04 \
  bash -lc '
    set -e
    export DEBIAN_FRONTEND=noninteractive
    apt-get update
    apt-get install -y --no-install-recommends \
      build-essential binutils zstd \
      cmake ninja-build \
      python3 python3-pip python3-venv python3-dev \
      libxcb1 libglib2.0-0 libgl1 libxext6 libxrender1 libsm6 libgomp1
    mkdir -p compile
    cd compile
    cmake -G Ninja ..
    ninja
  '

sudo chown -R "$USER:$USER" compile
```

The Docker dependencies include the XCB/OpenGL runtime required by OpenCV. Without them, the build can fail with `ImportError: libxcb.so.1`.

Successful output contains:

EXPECTED OUTPUT / REFERENCE

```
compile/output/generated_arch.arch/yolov8n_dla_m2m_compiled_640_384.bin
compile/output/generated_arch.arch/yolov8n-pose_dla_m2m_compiled_640_384.bin
compile/output/yolov8n_categories.txt
compile/output/yolov8n-pose_categories.txt
```

9. Install the models and reboot

Replace `BOARD_IP` with the address from `ifconfig eth0`.

HOST TERMINAL COMMANDS

```
cd ~/agilex5-camera-4kp30/source/yolo_cnn/compile/output

scp -r generated_arch.arch yolov8n_categories.txt yolov8n-pose_categories.txt \
  root@BOARD_IP:

ssh root@BOARD_IP \
  'sync && ls -l generated_arch.arch/*.bin yolov8n_categories.txt yolov8n-pose_categories.txt'

ssh root@BOARD_IP 'sync && reboot'
```

The destination is `/root` on the board and is stored on the SD-backed filesystem. Wait for the serial console login prompt, rediscover the DHCP address if necessary, and reload the web UI.

10. Final acceptance test

1. Confirm the DP monitor detects a signal and displays camera output.
2. Select camera 0 in the web UI and verify live video.
3. Select camera 1 and verify live video.
4. Select **Detect** and verify object boxes/labels.
5. Select **Pose** and verify pose overlays.
6. Reboot once more and confirm that both models and both camera sources remain usable.

With the FPGA AI Suite evaluation license, AI inference can stop after approximately 100,000 inferences. A full AI Suite license removes that limit.

Troubleshooting

Symptom	Check / corrective action
`quartus_pgm -l` shows no carrier	Recheck J35 USB cable, board power, host USB permissions, and the correct Programmer installation.
JTAG program fails	Confirm `S4 = OFF-OFF`, use the verified `top.core.jic`, and do not disconnect power during erase/program/verify.
Serial console is blank	Start the console before power-on; use the third port of the J2 four-port group; verify `S4 = ON-ON` and the microSD is seated.
`Permission denied` opening `/dev/ttyUSB*`	Add the host user to `dialout`, log out/in, or use `sudo` temporarily.
BusyBox rejects `ip -br`	Use `ifconfig eth0`.
`torch==2.6.0+cpu` has no matching version	The host Python is too new; use the Ubuntu 22.04 Docker command in section 8.
`ImportError: libxcb.so.1` in model build	Use the full Docker package list in section 8; it installs required OpenCV runtime libraries.
Display says “No signal”	Verify display input and DP cable, connect before board boot, reseat both ends, and power-cycle the monitor. Do not reflash QSPI solely for this symptom.
UI works but no DP output	At the HPS prompt run `pidof VvpIspDemo` and `systemctl --no-pager --full status vvp-isp.service`. Confirm the DP display link separately.

Official links and build inputs

Use the pinned `rel-25.1` resources when rebuilding or regenerating this specific design. The current `rel-26.1` documentation is useful for board guidance, but do not mix newer source, models, or generated artifacts with this `rel-25.1` image/JIC flow without revalidating the complete design.

Resource	Official link and when it is needed
Release-matched camera guide	https://altera-fpga.github.io/rel-25.1/embedded-designs/agilex-5/e-series/modular/camera/camera_4k_ai/camera_4k_ai/ — primary instructions for this asset release
Current camera guide	https://altera-fpga.github.io/rel-26.1/embedded-designs/agilex-5/e-series/modular/camera/camera_4k_ai/camera_4k_ai/ — current Altera documentation referenced by this project
Pre-built artifacts	https://github.com/altera-fpga/agilex-ed-camera-ai/releases/tag/rel-25.1 — download `top.core.jic` and `hps-first-vvp-isp-demo-image-agilex5_mk_a5e065bb32aes1.wic.gz`
Camera design source	https://github.com/altera-fpga/agilex-ed-camera-ai/tree/rel-25.1 — clone with `--recurse-submodules` for source build/model compilation
Source-build instructions	https://github.com/altera-fpga/agilex-ed-camera-ai/blob/rel-25.1/README.md — matching SOF and HPS-first RBF/JIC build flows

Resource	Official link and when it is needed
Quartus Prime Pro 25.1 Linux	https://www.intel.com/content/www/us/en/software-kit/851652/intel-quartus-prime-pro-edition-design-software-version-25-1-for-linux.html — required to rebuild the `rel-25.1` hardware design
Quartus download center	https://www.altera.com/products/development-tools/quartus — official portal for Programmer and device support
Nios V tools	https://www.altera.com/design/guidance/nios-v-developer — required by the source hardware build
FPGA AI Suite 2025.1	https://www.altera.com/downloads/add-development-tools/fpga-ai-suite-version-2025-1 — required by the model compiler
Modular Design Toolkit	https://github.com/altera-fpga/modular-design-toolkit — checked out automatically by the design source as a submodule
Linux SoC FPGA source	https://github.com/altera-opensource/linux-socfpga/tree/socfpga-6.6.22-lts — input to a custom Yocto software build
U-Boot SoC FPGA source	https://github.com/altera-opensource/u-boot-socfpga/tree/v2024.01 — input to a custom Yocto software build
Arm Trusted Firmware	https://github.com/ARM-software/arm-trusted-firmware/tree/socfpga_v2.11.0 — input to a custom Yocto software build
Yocto Project Poky	https://git.yoctoproject.org/poky — the matching software flow uses the `scarthgap` release
Ultralytics license	https://www.ultralytics.com/license — review before obtaining YOLO models
YOLOv8n detection model	https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n.pt — place in `source/yolo_cnn` before compiling
YOLOv8n pose model	https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n-pose.pt — place in `source/yolo_cnn` before compiling

Source-build choices

The downloadable JIC/SD-card flow in this runbook is the lowest-risk path for a first-time board bring-up. Rebuild only when the design must be changed.

- **FPGA-first SOF flow:** use `AGX_5E_Modular_Devkit_ISP_AI_FF_RD.xml` and the Yocto `kas/agilex_camera_ff.yml` configuration.
- **HPS-first RBF/JIC flow:** use `AGX_5E_Modular_Devkit_ISP_AI_RD.xml` and the Yocto `kas/agilex_camera.yml` configuration. This is the source-build equivalent of the pre-built QSPI + microSD boot approach.

Source builds require the correct license combination. The design README documents the OpenCore Plus, Video and Vision Processing Suite, Tone Mapping Operator, 3D LUT, FPGA AI Suite, MIPI D-PHY, MIPI CSI-2, and Nios V licensing constraints. The official example is a demonstration design and should be revalidated before production deployment.

Deployment record

Complete this section for each board that is prepared using this guide.

Field	Record
Date / operator	_____
Board serial number	_____
QSPI artifact and checksum verified	_____
microSD image and checksum verified	_____
Camera 0 live-video test	Pass / Fail — notes: _____
Camera 1 live-video test	Pass / Fail / Not fitted — notes: _____
Detection model test	Pass / Fail — notes: _____
Pose model test	Pass / Fail — notes: _____
DP display and web UI test	Pass / Fail — notes: _____

Handoff evidence

- Record the board DHCP address, monitor model, and camera serial numbers.
- Attach a screenshot of the web UI and a photograph of the DP overlay.
- Retain the release-asset checksums and the model compiler output with the board validation record.

Notes and corrective actions

_____

_____

_____

_____

Acceptance sign-off

Role	Name / signature / date
Bring-up engineer	_____
Reviewer	_____